

Mise en service de l'automatisation n8n

NetVerify — Vérification post-intervention & qualité de service · Tunisie Telecom

Compte rendu de configuration · Workflow « NetVerify — Contrôle de conformité J+14 » · 6 août 2026

1. Objet du document

Le workflow n8n `netverify-verification.json` orchestre le contrôle de conformité J+14 : il interroge le backend NetVerify, déclenche l'analyse des réclamations et diffuse le rapport par courriel. Trois portes d'entrée l'activent — la planification quotidienne de 06:00, la collecte horaire des mesures, et le bouton « Lancer la vérification » du tableau de bord Streamlit.

À l'import, le workflow ne fonctionnait pas de bout en bout. Ce document retrace les quatre blocages rencontrés, leur cause exacte et leur résolution, puis récapitule la configuration cible.

2. Synthèse des blocages traités

N°	Symptôme observé	Cause réelle	Résolution
1	« Waiting for you to call the Test URL » — le workflow reste en attente	Comportement normal : <i>Execute workflow</i> met le nœud webhook en écoute, il attend un appel HTTP	Appeler l'URL de test, ou tester les autres déclencheurs par <i>Execute step</i>
2	Le bouton du dashboard ne change rien dans n8n	Le dashboard appelle <code>/webhook/</code> (production), inexistant tant que le workflow n'est pas activé ; le front bascule alors silencieusement sur le backend	Activer le workflow, ou pointer temporairement le dashboard sur <code>/webhook-test/</code>
3	Aucun courriel n'est envoyé	Nœuds d'envoi neutralisés, et absence de credential SMTP	Credential SMTP Gmail avec mot de passe d'application
4	401 « Authentification requise »	Les nœuds HTTP appelaient les endpoints protégés sans en-tête d'authentification	Credential Header Auth <code>X-Service-Token</code>
5	503 « Aucun jeton de service n'est configuré »	L'en-tête arrivait bien, mais la variable <code>NETVERIFY_SERVICE_TOKEN</code> n'était pas définie côté backend	Définir la variable dans le terminal <i>avant</i> de lancer uvicorn

3. Blocage 1 — Le webhook en attente

Le bandeau « *Waiting for you to call the Test URL* » affiché sur le nœud **Déclenchement dashboard** n'est pas une erreur. Le workflow porte trois déclencheurs ; le bouton *Execute workflow* met le webhook en écoute et attend qu'un client appelle l'URL de test.

Ce que l'on veut tester	Manière de le déclencher
Le pipeline complet, comme le dashboard	Appeler <code>POST /webhook-test/netverify-verification</code> , ou cliquer le bouton du dashboard
Le contrôle planifié de 06:00	Clic droit sur Contrôle quotidien → <i>Execute step</i>
La collecte horaire	Clic droit sur Collecte horaire → <i>Execute step</i>

```
# PowerShell – appel manuel de l'URL de test
Invoke-RestMethod -Uri "http://localhost:5678/webhook-test/netverify-verification" `
  -Method Post -ContentType "application/json" -Body "{}"
```

Piège PowerShell. Sous Windows PowerShell 5.1, `curl` est un alias d' `Invoke-WebRequest` : la syntaxe `curl -X POST` échoue. Utiliser `Invoke-RestMethod`, ou `curl.exe` explicitement.

4. Blocage 2 — Le bouton du dashboard sans effet

Le dashboard n'appelle jamais l'URL de test. Il vise l'URL de production définie dans `frontend/netverify/settings.py` :

```
N8N_WEBHOOK_URL = os.getenv(
    "NETVERIFY_N8N_WEBHOOK", "http://127.0.0.1:5678/webhook/netverify-verification"
)
```

Or le chemin `/webhook/` n'existe que si le workflow est **activé**. Tant qu'il ne l'est pas, n8n renvoie un 404 que le front interprète comme « n8n indisponible » et compense en appelant directement le backend :

```
Bouton « Lancer la vérification »
└─▶ POST /webhook/netverify-verification 404 (workflow inactif)
    └─▶ repli : POST /verification/lancer 200 – le contrôle tourne, mais hors n8n
```

Ce repli est intentionnel : il garantit que l'interface reste utilisable même si l'automate est éteint, au prix des notifications. Mais il explique parfaitement le symptôme — un résultat s'affiche à l'écran, et pourtant le canvas n8n ne bouge pas.

Conséquence pratique. Une fois le workflow activé, les exécutions ne s'affichent plus en direct sur le canvas : elles se consultent dans l'onglet *Executions*. Pour observer le cheminement nœud par nœud pendant la mise au point, il faut pointer temporairement la variable `NETVERIFY_N8N_WEBHOOK` sur l'URL `/webhook-test/`.

5. Blocage 3 — L'envoi des courriels

Les trois nœuds d'envoi sont de type `n8n-nodes-base.emailSend`. Ils réclament une credential **SMTP** — et non une credential *Gmail OAuth2*, qui aurait imposé de créer un projet Google Cloud et un écran de consentement.

5.1 Mot de passe d'application Google

Depuis mai 2022, Google refuse l'authentification SMTP par mot de passe de compte. Un mot de passe d'application dédié est indispensable :

1. Activer la validation en deux étapes sur le compte Google — sans elle, la page des mots de passe d'application est inaccessible.
2. Générer un mot de passe sur `myaccount.google.com/apppasswords`, sous le nom « n8n NetVerify ».
3. Google affiche 16 lettres groupées par quatre. Les saisir **sans les espaces** ; ils ne sont plus jamais réaffichés.

5.2 Paramètres de la credential SMTP

Champ	Valeur
User	l'adresse Gmail complète
Password	les 16 caractères du mot de passe d'application, sans espaces
Host	<code>smtp.gmail.com</code>
Port	<code>465</code> — variante <code>587</code> avec SSL/TLS désactivé (STARTTLS)
SSL/TLS	activé
Nom de la credential	<code>SMTP NetVerify</code>

Erreur 535-5.7.8 BadCredentials. Elle vient de Google, jamais de n8n. Trois causes par ordre de fréquence : le mot de passe du compte a été saisi au lieu du mot de passe d'application ; des espaces ont été conservés au collage ; la validation en deux étapes a été désactivée après coup, ce qui révoque tous les mots de passe d'application existants.

L'adresse d'expédition, définie dans le nœud **Paramètres NetVerify**, doit être identique à celle du compte : Gmail rejette tout autre expéditeur.

Alternative pour la recette. Le service Mailtrap (offre gratuite) fournit un hôte SMTP jetable dont les messages atterrissent dans une boîte factice, avec le rendu HTML complet. Il évite d'envoyer de vrais courriels pendant la mise au point et ne demande aucun réglage Google.

6. Blocages 4 et 5 — Le jeton de service

Les endpoints d'automatisation `/mesures/rafraichir-tous` et `/mesures/verifier-conformite` sont protégés par la dépendance `automate_ou_superviseur` de `Backend/securite.py`. Elle accepte deux formes d'identité : un JWT de superviseur, ou un secret partagé présenté en en-tête `X-Service-Token`. n8n n'ayant pas d'interface de connexion, c'est la seconde voie qui s'applique.

La comparaison passe par `secrets.compare_digest`, afin de ne divulguer ni la longueur ni le préfixe du secret par le temps de réponse.

6.1 Lecture des codes de retour

Code	Signification	Correction
401	Aucun en-tête reçu : le nœud HTTP n'a pas de credential	Rattacher la credential Header Auth au nœud
503	En-tête reçu, mais aucun jeton n'est configuré sur le serveur	Définir <code>NETVERIFY_SERVICE_TOKEN</code> côté backend
429	Plafond de 6 appels par heure atteint	Attendre : bridage volontaire, pas une régression
200	Jeton valide, traitement exécuté	—

Cette distinction entre 401 et 503 est un choix de conception utile : elle sépare « l'automate ne s'annonce pas » de « le serveur n'a pas de secret à confronter », deux pannes de nature différente.

6.2 Procédure appliquée

Étape 1 — Générer le secret.

```
python Backend/gerer_utilisateurs.py jeton-service
```

La commande produit une chaîne aléatoire de 43 caractères (`secrets.token_urlsafe(32)`) et rappelle les deux endroits où la déclarer.

Étape 2 — Déclarer le secret côté backend. Le backend ne charge aucun fichier `.env` : il lit directement les variables d'environnement du processus. En lancement natif, la variable doit donc être définie dans le terminal *avant* le démarrage d'uvicorn.

```
# Ordre obligatoire : uvicorn lit l'environnement au démarrage,  
# et --reload ne recharge pas les variables.  
$env:NETVERIFY_SERVICE_TOKEN = "<le jeton généré>"  
python -m uvicorn main:app --reload
```

En déploiement conteneurisé, la variable transite par le fichier `.env` de la racine, que docker-compose interpole vers le service `backend`.

Étape 3 — Créer la credential dans n8n. Type *Header Auth*, champ *Name* = `X-Service-Token` (le nom de l'en-tête HTTP, non un libellé), champ *Value* = le jeton.

Étape 4 — Rattacher la credential. Sur les trois nœuds HTTP protégés — *Rafraîchir les mesures*, *Contrôle de conformité J+14*, *Collecte* — *rafraîchir les mesures* : *Authentication* → **Generic Credential Type** → *Generic Auth Type* → **Header Auth**. Le nœud *Santé du backend* reste sans authentification : il appelle `GET /`, seule route anonyme de l'API.

Étape 5 — Vérifier. Un appel volontairement muni d'un faux jeton distingue immédiatement les deux pannes :

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/mesures/rafraichir-tous" `
  -Method Post -Headers @{ "X-Service-Token"="faux" }
```

Un **503** signale que la variable n'a pas atteint le backend ; un **401** confirme au contraire qu'un jeton est bien configuré et que seul celui présenté est erroné — c'est le résultat attendu.

Cause du 503 rencontré. La variable avait été définie dans un terminal, et uvicorn lancé depuis un autre. Une variable posée par `SetEnvironmentVariable(..., "User")` ne s'applique qu'aux terminaux ouverts *ensuite* ; celui déjà ouvert conserve son environnement d'origine.

7. Configuration cible

7.1 Credentials n8n

Type	Nom	Nœuds concernés
Header Auth	NetVerify – Jeton de service	Rafraîchir les mesures · Contrôle de conformité J+14 · Collecte — rafraîchir les mesures
SMTP	SMTP NetVerify	Rapport d'anomalies · Synthèse quotidienne · Alerte technique

7.2 Variables d'environnement

Variable	Portée	Rôle
NETVERIFY_SERVICE_TOKEN	backend	Secret partagé avec n8n ; sans lui, aucun automate n'est admis
NETVERIFY_SECRET_KEY	backend	Signature des JWT ; si absente, une clé aléatoire est tirée à chaque démarrage et toutes les sessions sautent
NETVERIFY_DATABASE_URL	backend	Connexion à la base ; si absente, repli sur la base de développement locale
NETVERIFY_N8N_WEBHOOK	frontend	URL du webhook visée par le bouton du dashboard

7.3 Adresse du backend vue depuis n8n

Le paramètre `api_base_url` du nœud **Paramètres NetVerify** dépend du mode de lancement de n8n :

n8n s'exécute...	Valeur de <code>api_base_url</code>
en natif	<code>http://127.0.0.1:8000</code>
en conteneur	<code>http://host.docker.internal:8000</code>

Depuis un conteneur, `127.0.0.1` désigne n8n lui-même et non le backend : c'est la cause d'échec la plus fréquente au premier démarrage.

8. Points de vigilance

- **Plafond de six appels par heure** sur les endpoints de traitement de masse. Des essais répétés provoquent un 429, sans que rien ne soit cassé.
- **L'écoute de l'URL de test est à usage unique.** Il faut cliquer *Execute workflow* avant chaque essai.
- **Un seul workflow actif par chemin de webhook.** Si un ancien workflow revendique `netverify-verification`, le désactiver au préalable.
- **Le jeton de service ne doit figurer ni dans le dépôt, ni dans un rapport.** Il se régénère à volonté par la commande de l'étape 1.

- **Le dashboard reçoit toujours un 200**, succès comme échec. Un 5xx ferait basculer le front sur l'appel direct au backend et rejouerait un contrôle déjà passé, dupliquant les journaux de suivi.

9. Résultat

Chaîne complète opérationnelle. Le déclenchement depuis le tableau de bord atteint n8n, qui vérifie la santé du backend, rafraîchit les mesures, exécute le contrôle J+14, met en forme le rapport et l'expédie par courriel, avant de renvoyer le compte rendu au dashboard. Les trois portes d'entrée — planification quotidienne, collecte horaire, appel manuel — partagent ce même pipeline.

Document généré le 6 août 2026 — projet NetVerify. Les secrets (jeton de service, mot de passe d'application SMTP) sont volontairement omis de ce document et doivent être conservés hors du dépôt.